

IoT Project Report: Plant Watering and Monitoring System

Contributors:

Rooble Dahir

Samuel Kwok

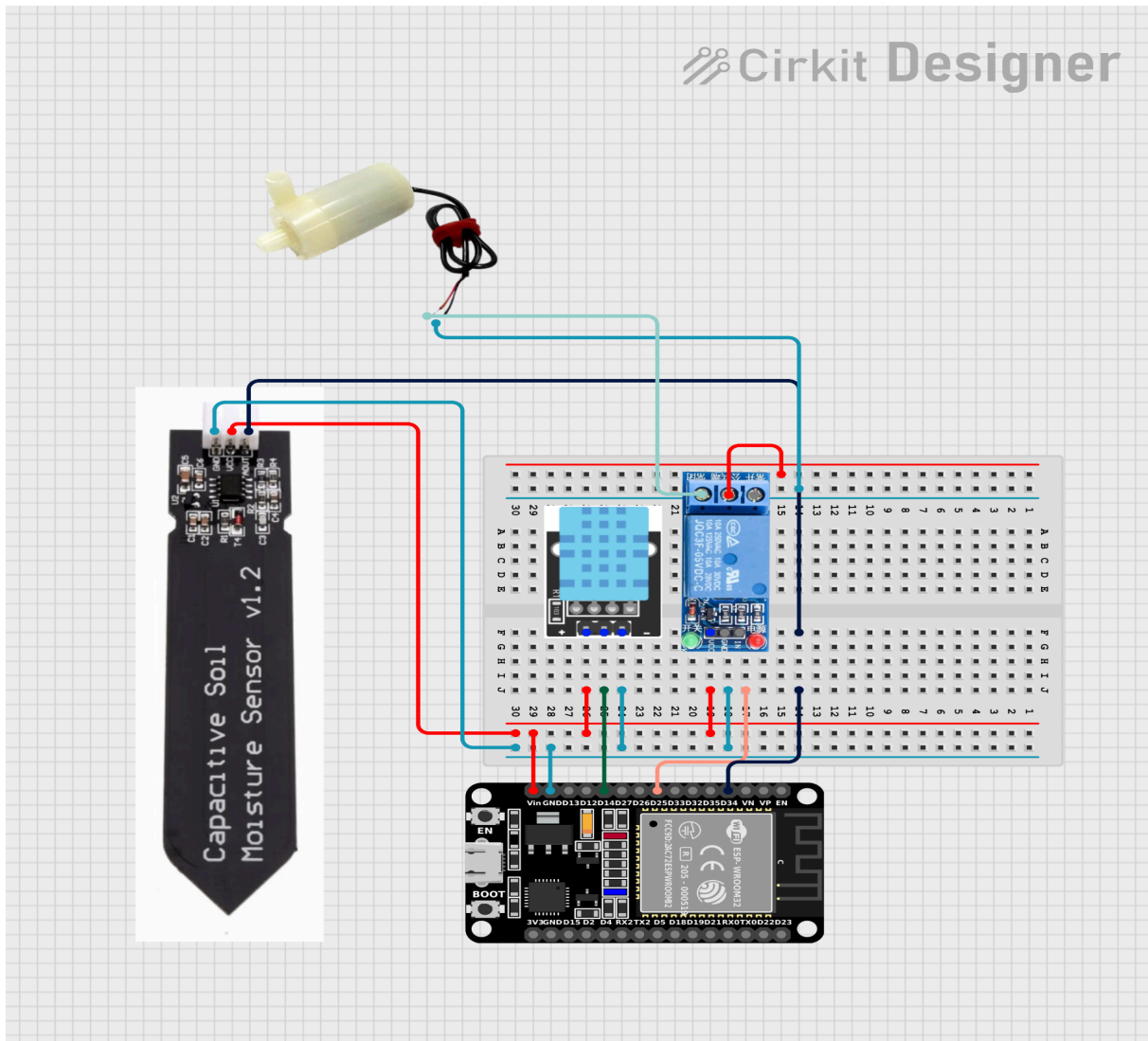
Tommy Long

Introduction.....	2
Circuit Diagram.....	2
Components Used.....	3
Principle of Operation.....	3
Code.....	3
Blynk Integration.....	5
Sample Results.....	6
Challenges.....	6
Conclusions.....	6
References.....	7

Introduction

Taking care of your plants placed in and around your home can be difficult when you're balancing multiple tasks at once, or you aren't at home. The purpose of our project is to address this issue with an IoT-based watering system that can monitor the soil's moisture level, humidity, and temperature. By using a small water pump, sensors, and the Blynk app, we can water the plant automatically or manually from our preferred device. This allows us to provide consistent care for the plant, all while monitoring vital conditions from anywhere at any time.

Circuit Diagram



Components Used

- ESP32 Development Board & PSU
- Soil Moisture Sensor
- Breadboard and wires
- DHT11 Temperature and Humidity Sensor
- Relay Module
- 5V Mini Water Pump
- Plastic Tubing

Principle of Operation

The operation of the automatic watering system works in 2 ways: the pump can be activated either via the capacitive soil sensor to detect moisture in the soil, and it can also be manually triggered via Blynk. The ESP32 polls the capacitive moisture sensor to detect if the soil is dry, and if it is, then it will activate the pump until the soil sensor detects it is sufficiently moist, and this functionality does not require an internet connection.

You can alternatively connect via Blynk to monitor the temperature and humidity of the room the plant is in, and trigger the water pump if need be.

Code

```
// Setup Blynk
#define BLYNK_TEMPLATE_ID "TMPL2CJ396PL2"
#define BLYNK_TEMPLATE_NAME "Irrigation System"
#define BLYNK_AUTH_TOKEN "[REDACTED]"
// Include libraries for features
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

// WiFi credentials
char ssid[] = "[REDACTED]";
char pass[] = "[REDACTED]";

// Configure pins for sensors/relay:
// Relay pin
#define RELAY_PIN 25
// Soil sensor pin
#define SOIL_SENSOR_PIN 34
// DHT sensor
#define DHTPIN 14 // GPIO connected to DHT11 data
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

BlynkTimer timer;
```

```

// Manual pump control from Blynk button (V0)
BLYNK_WRITE(V0) {
  int pinValue = param.asInt();
  digitalWrite(RELAY_PIN, pinValue);
}

// Soil moisture reading
void checkSoil() {
  int soilValue = analogRead(SOIL_SENSOR_PIN);
  int soilPercent = map(soilValue, 4095, 0, 0, 100);
  soilPercent = constrain(soilPercent, 0, 100);
// Print readings in serial for monitoring/troubleshooting
  Serial.print("Soil Moisture: ");
  Serial.print(soilPercent);
  Serial.println("%");
// Write moisture value to Blynk for GUI
  Blynk.virtualWrite(V1, soilPercent);
// Turns on pump if soil is too dry
  if (soilPercent < 30) {
    digitalWrite(RELAY_PIN, HIGH);
    Serial.println("Soil is dry! Pump ON");
  } else {
    digitalWrite(RELAY_PIN, LOW);
    Serial.println("Soil is moist. Pump OFF");
  }
}

// DHT sensor reading & Blynk update
void sendDHT() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
// Throw an error if sensor is not working
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor");
    return;
  }
// Prints values to serial to monitor
  Serial.print("Temp: ");
  Serial.print(temperature);
  Serial.print("°C, Humidity: ");
  Serial.print(humidity);
  Serial.println("%");

  Blynk.virtualWrite(V2, humidity); // V2 for humidity
  Blynk.virtualWrite(V3, temperature); // V3 for temperature
}

// Setup pin modes and enables serial for monitoring
void setup() {
  Serial.begin(115200);

```

```

pinMode(RELAY_PIN, OUTPUT);
digitalWrite(RELAY_PIN, LOW);

dht.begin();

Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

timer.setInterval(3000L, checkSoil);
timer.setInterval(3000L, sendDHT);
}
// Runs script on loop
void loop() {
  Blynk.run();
  timer.run();
}

```

Blynk Integration

The system was fully integrated with the Blynk IoT Cloud, which allowed us to monitor the sensors and control the irrigation system remotely. The ESP32 continuously sends real-time soil moisture, temperature, and humidity readings to the mobile app dashboard, which displays it using gauges. The dashboard also has a manual switch that allows us to turn the water pump on or off. To set up Blynk, a template was created and within the template we configured the datastreams to match the virtual pins in our code. After that, we created a device which generated a unique authentication token that is used to link our ESP32.

New Device Created!

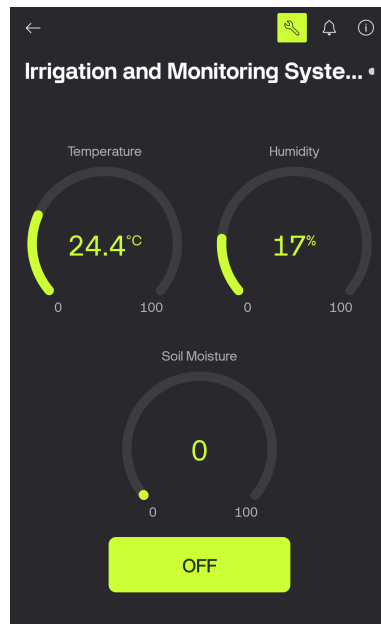
```

#define BLYNK_TEMPLATE_ID "TMPL2vzM0k5BP"
#define BLYNK_TEMPLATE_NAME "test"
#define BLYNK_AUTH_TOKEN "YKwnfETTBKK_RrS_efLaqE6WUsvULMdH"

```

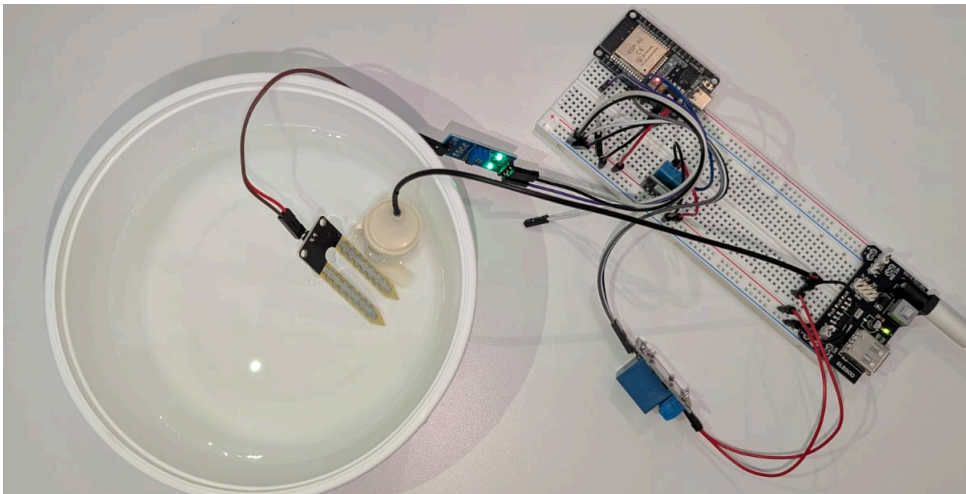
Datastreams

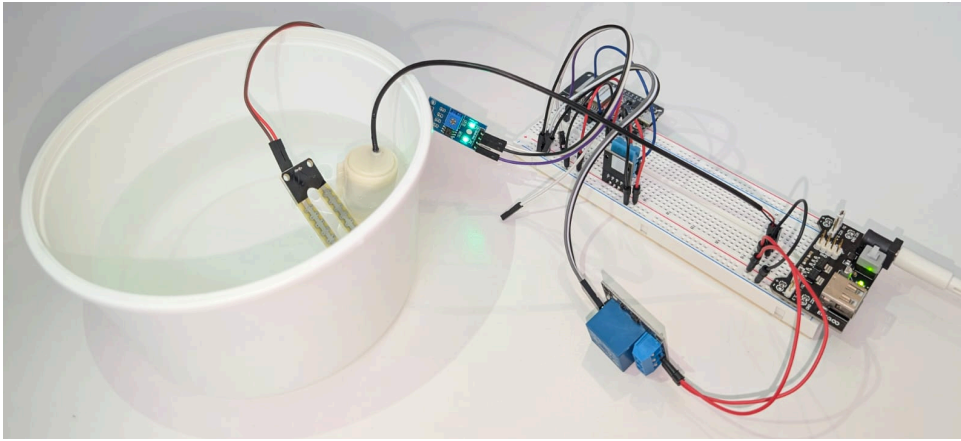
ID	Name	Pin	Color	Data Type	Units	Is Raw	Min	Max	Decimals	Default Value	Automation Type
2	Irrigation	V1		Integer		false	0	100	--	0	Switch
1	Water Pump	V0		Integer		false	0	1	--	0	Switch
3	Temp	V3		Double	°C	false	0	100	###		Sensor
4	Humidity	V2		Double	%	false	0	100	#		Sensor



Sample Results

For our testing, we did not have soil, so instead we collected the soil moisture level by putting it in the water reservoir. During the testing, the irrigation system performed exactly as intended. The ESP32 collected soil moisture, temperature, and humidity readings every 3 seconds and sent the info to the Blynk dashboard, where it would be displayed. The automatic irrigation also worked correctly. When the soil moisture dropped below 30%, the relay module activated and the water pump was turned on, and once the level exceeded 30%, the pump would shut off. Our manual pump switch also responded without delay, confirming the successful integration of our system.





Challenges and Future Improvements

Integrating the pump into the circuit was difficult as the relay included in the kit was just a 5V relay, and not a module that can be triggered by 3.3V, which is what the ESP32 outputs. We ended up needing to get a relay module board to be able to trigger the relay to activate the pump, but otherwise the project went smoothly.

For future applications, there are several improvements that could enhance the capabilities of the system. Adding other sensors like light intensity and soil pH could give a more detailed report of the plant's health. An automatic fertilizer dispenser and water level sensor for the reservoir would help further automate plant care. These upgrades would make the system more efficient and autonomous for a longer period of time.

Conclusions

Overall, the project ultimately worked as intended, highlighting the utility of IoT not only for large-scale/complex projects but also for everyday tasks. The system was able to read sensor data and forward it to the Blynk app, and activate it when needed. We did run into a few problems with the pump and relay; however, we were able to fix them through some testing, adjustments and replacements. Our final result ended up being an easy-to-use, practical system that can work both automatically and manually from anywhere at any time.

References/Resources

Arduino IDE. <https://www.arduino.cc/en/software/>

Blynk IoT Platform. <https://www.blynk.io>